

Question Answering using Knowledge Graphs

Présenter par:

- Touzani Yassine
- Sabkari Abderrahmane

01. Introduction

IDÉE GÉNÉRALE DU PROJET

Le projet consiste à créer un système capable de répondre automatiquement à des questions en utilisant un graphe de connaissances, qui relie les entités et leurs relations pour fournir des réponses précises et compréhensibles

- **EXEMPLE**

what is the capital of Morocco ? → Rabat

01. Introduction

IDÉE GÉNÉRALE DU PROJET

Le projet consiste à créer un système capable de répondre automatiquement à des questions en utilisant un graphe de connaissances, qui relie les entités et leurs relations pour fournir des réponses précises et compréhensibles

- **EXEMPLE**

what is the capital of Morocco ?

1. Marrakech
2. Safi
3. Rabat

01. Introduction

DIFFÉRENTES APPROCHES DU PROJET

Il y a deux approches pour atteindre l'objectif :

- Pour les questions à choix multiple, un **réseaux de neurones en graphes (GNN)** est utilisé pour analyser les relations entre les mots et déterminer la meilleure réponse.
- Pour les questions à réponse directe, la question est convertie en requête SPARQL puis exécutée sur une base de données de graphe comme **Wikidata ou DBpedia** pour obtenir la réponse.

02. QA base sur gnn

PROCESSUS DE CONSTRUCTION DU MODÈLE

Dataset utilise :

Nous avons utilisé le dataset **CommonsenseQA**, qui contient des questions à choix multiple basées sur des connaissances de sens commun.

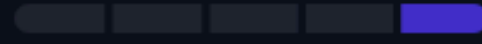
contient 9740 questions à choix multiple.

https://huggingface.co/datasets/tau/commonsense_qa

02. QA base sur gnn

PROCESSUS DE CONSTRUCTION DU MODÈLE

structure de Dataset :

question string · <i>lengths</i>	question_concept string · <i>lengths</i>	choices dict	answerKey string · <i>classes</i>
 89↔126 16.6%	 9↔12 17.3%		 A 19.6%
The sanctions against the school were a punishing blow, and they seemed to what the efforts the school had made to change?	punishing	<pre>{ "label": ["A", "B", "C", "D", "E"], "text": ["ignore", "enforce", "authoritarian", "yell at", "avoid"] }</pre>	A

02. QA base sur gnn

PROCESSUS DE CONSTRUCTION DU MODÈLE

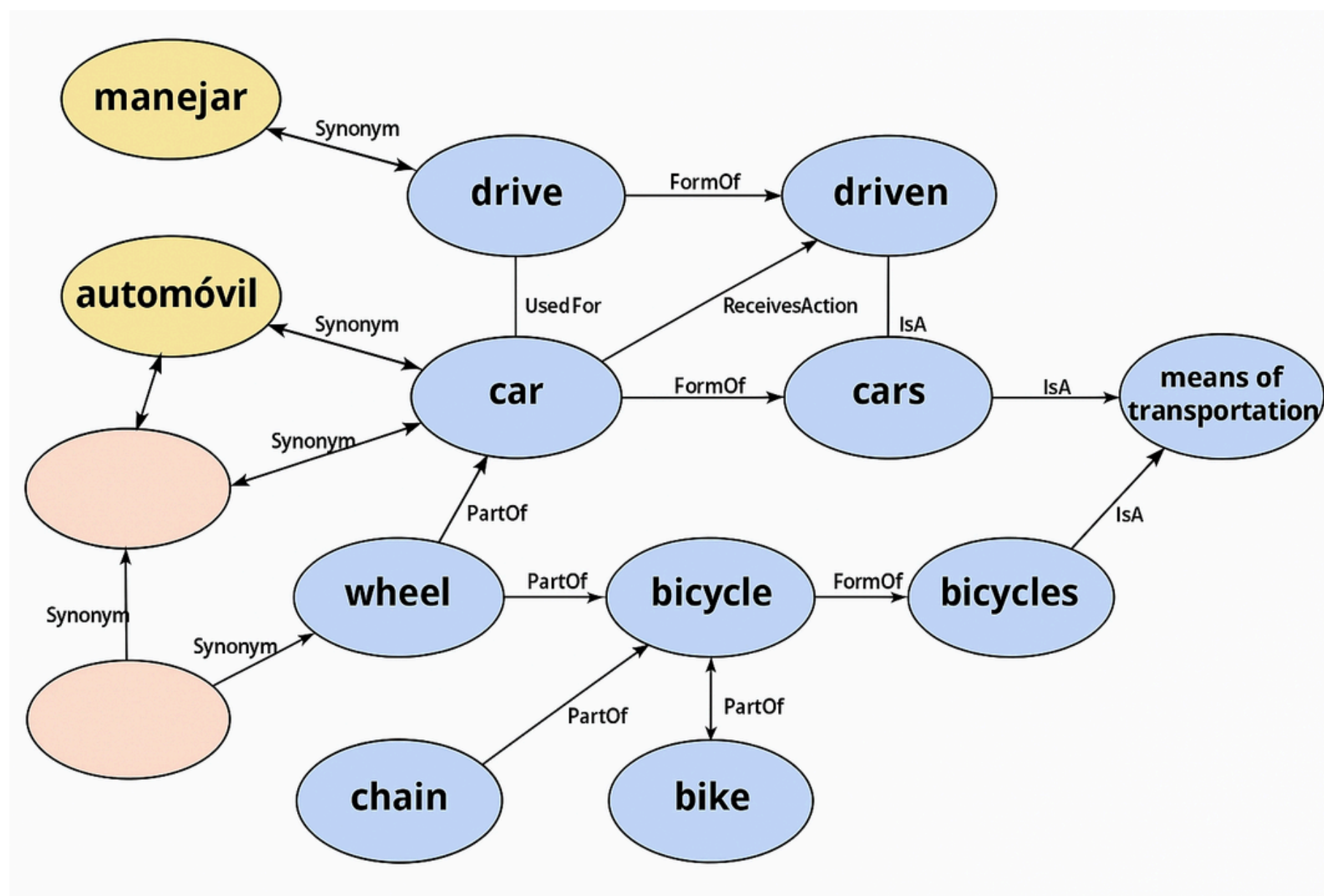
Base de connaissances : ConceptNet-Assertions 5.6.0

La base de connaissances ConceptNet-Assertions 5.6.0 est un graphe de connaissances qui relie des concepts entre eux par des relations sémantiques

02. QA base sur gnn

PROCESSUS DE CONSTRUCTION DU MODÈLE

Base de connaissances : ConceptNet-Assertions 5.6.0



02. QA base sur gnn

PROCESSUS DE CONSTRUCTION DU MODÈLE

Etape 1 : Extraction du sous-graphe

- Chaque question sera soumise à un prétraitement comprenant une tokenisation, suivie d'une sélection des tokens correspondant uniquement aux catégories NOUN (nom), PROPN (nom propre) et VERB (verbe).

```
text = "Cats are chasing Mice"
→ tokens: ['cats', 'are', 'chasing', 'mice']
→ pos_:   ['NOUN', 'AUX', 'VERB', 'NOUN']
→ lemmas: ['cat', 'be', 'chase', 'mouse']
```

02. QA base sur gnn

PROCESSUS DE CONSTRUCTION DU MODÈLE

Etape 1 : Extraction du sous-graphe

- Prétraitement de la base de connaissances ConceptNet

```
conceptnet_df = load_conceptnet_56(limit=1_000_000)
conceptnet_df
```

	start	end	rel
8809	0	1	Antonym
8810	12 hour clock	24 hour clock	Antonym
8811	24 hour clock	12 hour clock	Antonym
8812	5	3	Antonym
8813	a.c	d.c	Antonym

02. QA base sur gnn

PROCESSUS DE CONSTRUCTION DU MODÈLE

Etape 1 : Extraction du sous-graphe

- Prétraitement de la base de connaissances ConceptNet

DGL (Deep Graph Library) est une bibliothèque Python open-source conçue pour faciliter le développement et l'entraînement de réseaux de neurones sur des graphes (Graph Neural Networks — GNNs).

```
g = dgl.graph((src, dst))

g.ndata["name"] = torch.arange(len(nodes))

rel_types = list(df["rel"].unique())
rel2id = {r: i for i, r in enumerate(rel_types)}
edge_rel = torch.tensor([rel2id[r] for r in df["rel"]])
g.edata["rel_type"] = edge_rel
```

02. QA base sur gnn

PROCESSUS DE CONSTRUCTION DU MODÈLE

Etape 1 : Extraction du sous-graphe

- creation de sous-graphe
 1. Extraire les arêtes de chaque concept : pour chaque concept d'intérêt, on récupère toutes ses relations présentes dans ConceptNet.
 2. Vérifier les connexions entre concepts : on regarde si chaque concept est relié aux autres concepts du groupe.
 3. Construire le sous-graphe : après avoir extrait toutes les arêtes reliant ces concepts, on crée un sous-graphe spécifique à ces concepts à partir du graphe complet de ConceptNet.

```
sub_nodes = list(neighbors)
subgraph = concept_graph.subgraph(sub_nodes)
```

02. QA base sur gnn

PROCESSUS DE CONSTRUCTION DU MODÈLE

Etape 2 : preparation de model

- **GNN - Graph Neural Network**

1. Un GNN (Graph Neural Network) est un réseau de neurones conçu pour apprendre à partir de données structurées en graphe.
2. Chaque nœud a des features (un vecteur de caractéristiques, souvent un embedding).
3. Le GNN apprend à mettre à jour les embeddings des nœuds en tenant compte de leurs voisins dans le graphe.
4. GraphSAGE (Graph Sample and Aggregate) est un GNN simple et efficace qui utilise une agrégation par moyenne.

02. QA base sur gnn

PROCESSUS DE CONSTRUCTION DU MODÈLE

Etape 2 : preparation de model

- **GNN - Graph Neural Network**

Le module GNNGraphSAGE traite les embeddings générés par RoBERTa pour les concepts du graphe, en les combinant avec les informations du sous-graphe correspondant.

Cela permet d'obtenir des embeddings enrichis par les relations entre les concepts du graphe.

02. QA base sur gnn

PROCESSUS DE CONSTRUCTION DU MODÈLE

Etape 2 : preparation de model

- preparation des donnees pour obtenir le score final
 1. Nous allons concaténer chaque question avec l'une de ses réponses possibles (par exemple en sélectionnant une réponse parmi les choix), puis utiliser le modèle RoBERTa pour générer un embedding représentatif de ce pair question-réponse.
 2. Ensuite, nous allons générer un embedding représentatif du graphe en calculant la moyenne de tous les embeddings des nœuds du graphe

02. QA base sur gnn

PROCESSUS DE CONSTRUCTION DU MODÈLE

Etape 2 : preparation de model

- **Fusionner la compréhension du langage (RoBERTa) et la connaissance (graphe).**

On concatène l'embedding textuel de la paire question–réponse (RoBERTa) avec l'embedding global du sous-graphe (GNN).

- **prédire la meilleure réponse**

Ce vecteur combiné passe ensuite dans une couche linéaire (nn.Linear) qui agit comme un classifieur.

Cette couche apprend des poids qui déterminent comment pondérer les informations du texte et du graphe.

Le résultat est un score pour chaque réponse possible.

02. QA base sur gnn

PROCESSUS DE CONSTRUCTION DU MODÈLE

Etape 3 : Entraînement du model

- boucle d'entraînement

```
for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0

    pbar = tqdm(train_loader, desc=f"Epoch {epoch+1}/{num_epochs}")
    for batch_idx, batch in enumerate(pbar):
        optimizer.zero_grad()

        # Move graph and features to device
        question = batch[0]['question'] # batch_size=1
        answers = batch[0]['answers']
        subgraph = batch[0]['graph'].to(device)
        node_features = batch[0]['node_features'].to(device)
        label = torch.tensor([batch[0]['label']]).to(device)
```

02. QA base sur gnn

PROCESSUS DE CONSTRUCTION DU MODÈLE

Etape 3 : Entraînement du model

- boucle d'entraînement

```
# Forward pass
scores = model(question, answers, subgraph, node_features)
scores = scores.unsqueeze(0) # [batch_size=1, num_choices]

loss = criterion(scores, label)
loss.backward()
optimizer.step()

running_loss += loss.item()
pbar.set_postfix({'loss': running_loss / (batch_idx + 1)})

print(f"Epoch {epoch+1} finished. Average loss: {running_loss / len(train_loader):.4f}")
```

02. QA base sur gnn

PROCESSUS DE CONSTRUCTION DU MODÈLE

Etape 3 : Entraînement du model

- **boucle d'entraînement**

1. `loss.backward()` calcule les gradients pour tous les poids du modèle (RoBERTa, GNN, Linear).
2. L'optimiseur Adam ajuste les poids selon ces gradients :
3. RoBERTa apprend à mieux encoder le texte,
4. GNN apprend à mieux utiliser les relations du graphe,
5. Linear classifier apprend à combiner les deux pour produire des scores précis.
6. Ce processus est répété pour chaque batch et sur plusieurs epochs.

02. QA base sur gnn

PROCESSUS DE CONSTRUCTION DU MODÈLE

Etape 3 : Entraînement du model

- boucle d'entraînement

```
torch.save(model.state_dict(), "qa_gnn_model.pt")  
print("Model saved to qa_gnn_model.pt")
```

Python

```
Epoch 1/3: 0%|          | 0/9741 [00:00<?, ?it/s]/opt/conda/lib/python3.10/site-packages/dgl/backend/pytorch/tensor.py:445: UserWarning: TypedStorage  
  assert input.numel() == input.storage().size(), (  
Epoch 1/3: 0%|          | 4/9741 [00:45<30:55:09, 11.43s/it, loss=1.6]
```

```
OutOfMemoryError: CUDA out of memory. Tried to allocate 3.06 GiB (  
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell
```

03. KGQA sur DBpedia (Question Answering sur Graphes de Connaissances)

IDÉE GÉNÉRALE DU PROJET

Principe:

L'utilisateur pose une question :

« Who painted the Mona Lisa? »

03. KGQA sur DBpedia (Question Answering sur Graphes de Connaissances)

IDÉE GÉNÉRALE DU PROJET

Principe:

Le système transforme la phrase en requête SPARQL sur DBpedia

03. KGQA sur DBpedia (Question Answering sur Graphes de Connaissances)

IDÉE GÉNÉRALE DU PROJET

Principe:

DBpedia renvoie la réponse structurée (Leonardo da Vinci).

03. C'est quoi DBpedia ?

DBPEDIA & LES REQUÊTES SPARQL

DBpedia = base de connaissances construite à partir de Wikipédia

03. C'est quoi DBpedia ?

DBPEDIA & LES REQUÊTES SPARQL

Les infos sont stockées sous forme de triplets RDF :

Sujet – Prédicat – Objet

03. C'est quoi DBpedia ?

DBPEDIA & LES REQUÊTES SPARQL

Exemple:

Paris – est capitale de – France

03. C'est quoi DBpedia ?

DBPEDIA & LES REQUÊTES SPARQL

Exemple de requête SPARQL (Méthode 1):

Query Text

```
select distinct ?uri where {<http://dbpedia.org/resource/Mona_Lisa> dbo:author ?uri}
```

↑
Sujet

↑
Prédicat

↑
Objet

03. C'est quoi DBpedia ?

DBPEDIA & LES REQUÊTES SPARQL

Exemple de requête SPARQL (Méthode 2):

Query Text

```
select distinct ?uri where { ?w dbo:author ?uri . ?w rdfs:label "Mona Lisa"@en }
```

↑ ↑ ↑ ↑
Sujet **Prédictat** **Objet** ?w = Rechercher le lien du sujet

C'est quoi DBpedia ?

DBPEDIA & LES REQUÊTES SPARQL

Exemple de requête SPARQL (Méthode 2):

Query Text

```
select distinct ?uri where {?w dbo:author ?uri . ?w rdfs:label "Mona Lisa"@en}
```

↑ ↑ ↑ ↑

Sujet Prédicat Objet ?w=

<http://dbpedia.org/resource/Mona_Lisa>

03. C'est quoi DBpedia ?

DBPEDIA & LES REQUÊTES SPARQL

Exemple de requête SPARQL (Méthode 2):

Query Text

```
select distinct ?uri where {<http://dbpedia.org/resource/Mona_Lisa> dbo:author ?uri}
```

↑
Sujet

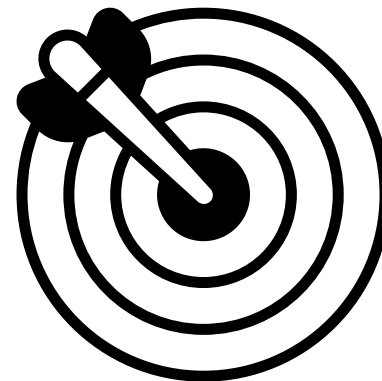
↑
Prédicat

↑
Objet

03. C'est quoi DBpedia ?

DBPEDIA & LES REQUÊTES SPARQL

But : récupérer automatiquement des données (lieux, personnes, films, etc.)



03. Baser sur l'architecture d'un article

Expert Systems With Applications

Expert Systems With Applications 229 (2023) 120383



Contents lists available at [ScienceDirect](https://www.sciencedirect.com)

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa



SPARQL-QA-v2 system for Knowledge Base Question Answering

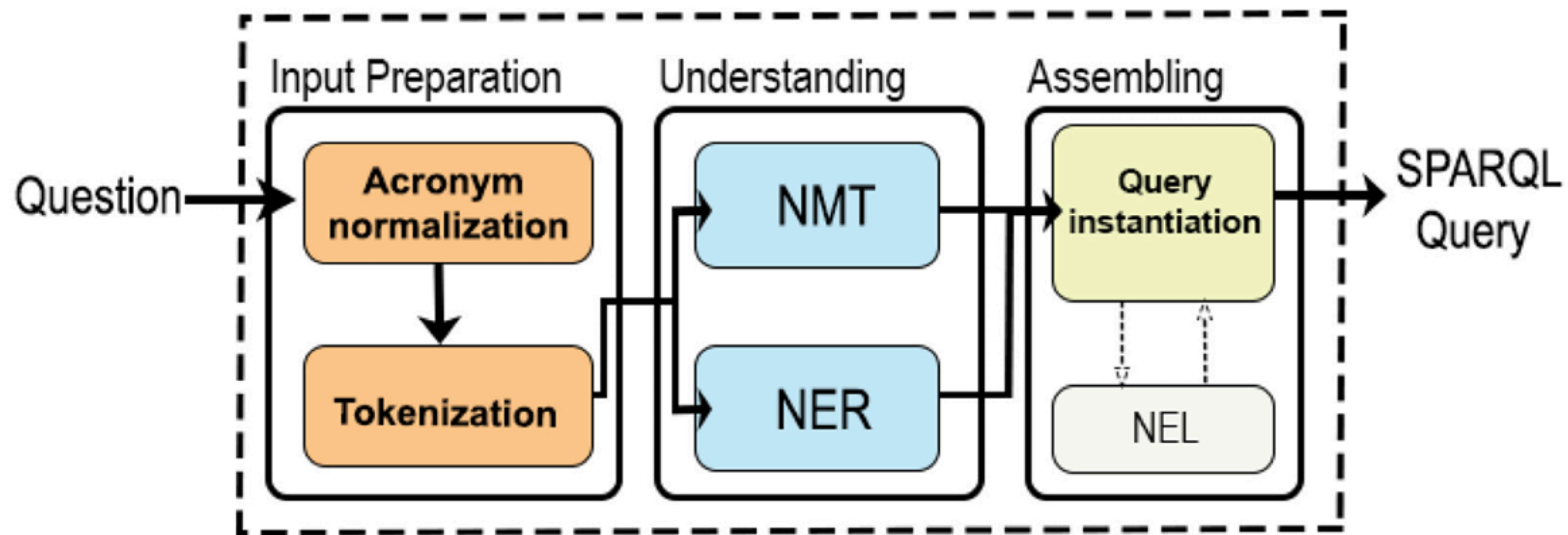
Manuel A. Borroto*, Francesco Ricca

Department of Mathematics and Computer Science, University of Calabria, Via Pietro Bucci, Cubo 30B, Rende, 87036, Calabria, Italy



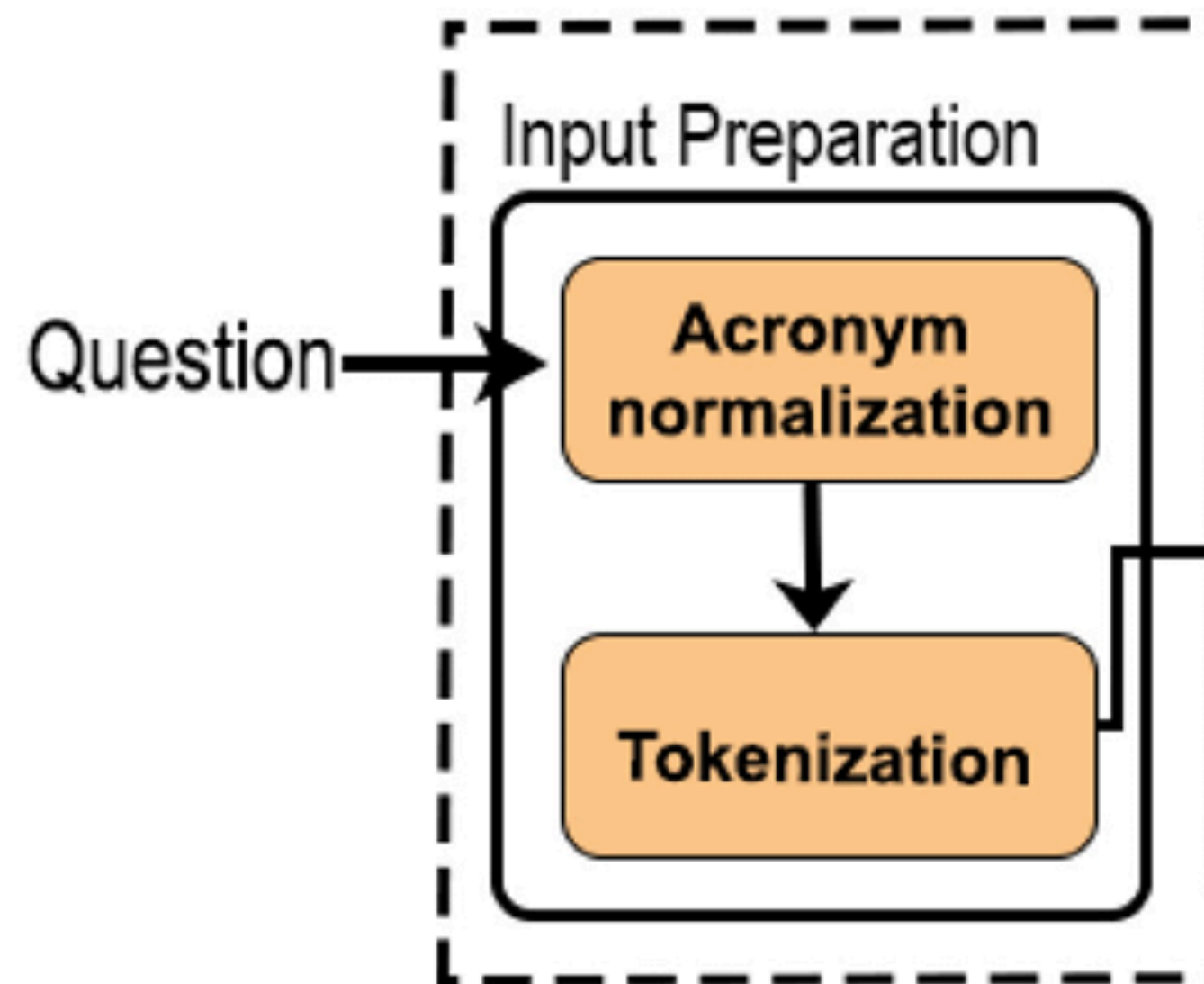
04. Architecture

Architecture pour traduire une question à une requête SPARQL.



04. 1. Préparation de l'entrée (Input Preparation)

Cette première étape vise à nettoyer et enrichir la question en langage naturel.



04. 1. Préparation de l'entrée (Input Preparation)

Normalisation des acronymes

But : Remplacer les acronymes par leurs noms complets pour mieux identifier les entités.



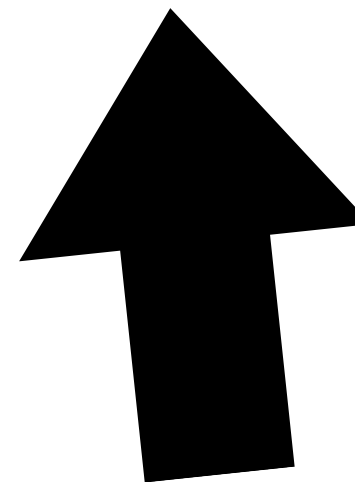
04. 1. Préparation de l'entrée (Input Preparation)

Normalisation des acronymes

Exemple:

Question originale :

« How large is the area of **UK** ? »



04. 1. Préparation de l'entrée (Input Preparation)

Normalisation des acronymes

Exemple:

Question originale :

spaCy détecte « **UK** » comme une entité.
Country Converter transforme « **UK** » → **United Kingdom**.

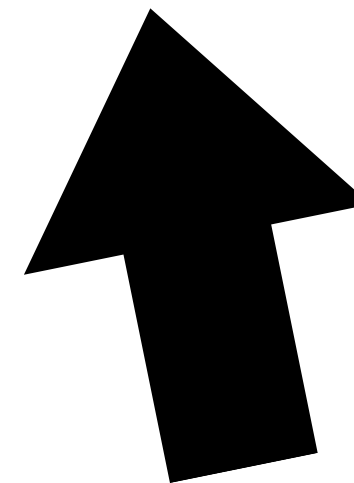
04. 1. Préparation de l'entrée (Input Preparation)

Normalisation des acronymes

Exemple:

Question transformée :

« How large is the area of **United Kingdom** ? »



04. 1. Préparation de l'entrée (Input Preparation)

Normalisation des acronymes

But:

Cela améliore la capacité à trouver
dbr:United_Kingdom dans DBpedia.

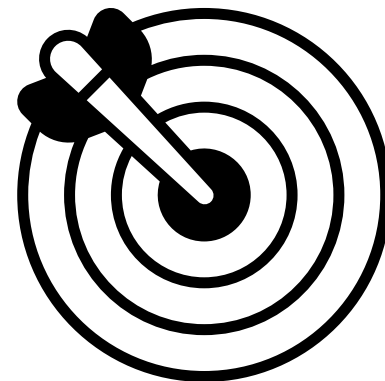


04. 1. Préparation de l'entrée (Input Preparation)

Tokenisation

But:

La question est découpée en tokens (mots) puis convertie en embeddings fastText.



04. 1. Préparation de l'entrée (Input Preparation)

Tokenisation

Exemple:

Tokenisation :

["Who", "painted", "the", "Mona", "Lisa"]

04. 1. Préparation de l'entrée (Input Preparation)

Représentation finale d'entrée pour le réseau

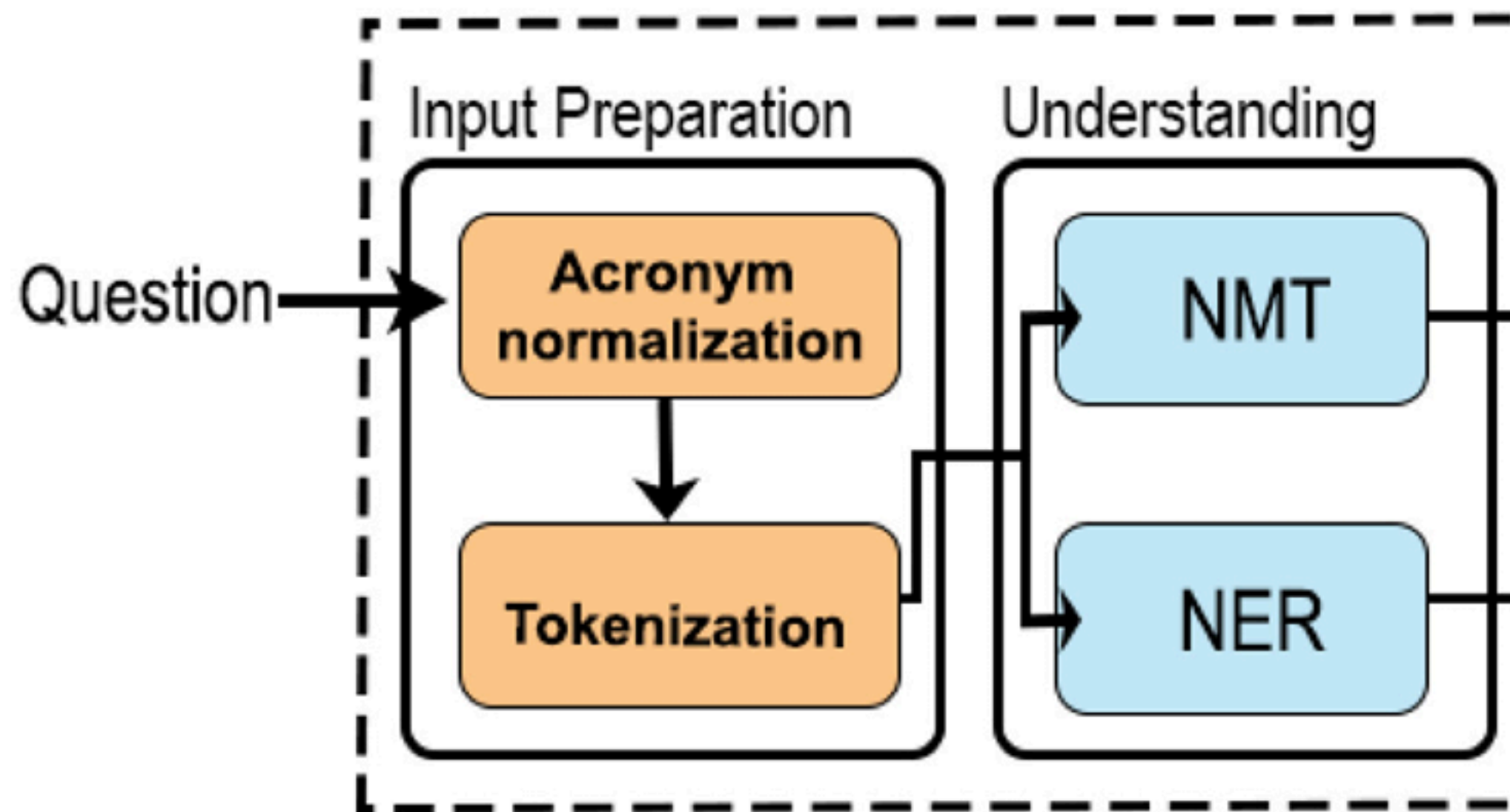
Exemple:

Les vecteurs sont empilés dans une matrice :

```
[  
  [0.12, -0.03, 0.55, ..., 0.01], ← embedding("Who")  
  [0.08, 0.41, -0.22, ..., 0.33], ← embedding("painted")  
  [-0.02, 0.11, 0.07, ..., 0.02], ← embedding("the")  
  [0.44, -0.12, 0.65, ..., -0.04], ← embedding("Mona")  
  [0.39, -0.09, 0.61, ..., -0.02] ← embedding("Lisa")  
]
```

04. 2. Compréhension de la séquence (Sequence Understanding)

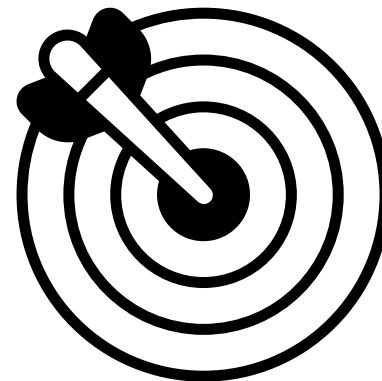
Deux modèles : NMT et NER.



04. 2. Compréhension de la séquence (Sequence Understanding)

A. NMT (Neural Machine Translation).

But : Transforme la question en un template SPARQL avec placeholders.



04.

2. Compréhension de la séquence (Sequence Understanding)

A. NMT (Neural Machine Translation).

Exemple:

Question originale :

« Who painted the Mona Lisa ? »

04.

2. Compréhension de la séquence (Sequence Understanding)

A. NMT (Neural Machine Translation).

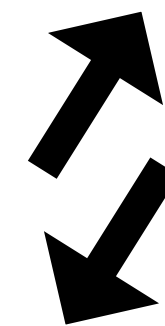
Exemple:

NMT génère :

```
SELECT ?a WHERE { ?w dbo:author ?a . ?w rdfs:label $1 }
```

Ici:

- ?w = variable pour l'entité recherchée
- \$1 = emplacement où mettre la valeur « **Mona Lisa** »



04.

2. Compréhension de la séquence (Sequence Understanding)

B. NER (Named Entity Recognition)

Identifie les entités dans la question

04.

2. Compréhension de la séquence (Sequence Understanding)

B. NER (Named Entity Recognition)

Exemple:

produit (BIO) :

Who(0) painted(0) the(0) Mona(**B**) Lisa(**I**)

04. 2. Compréhension de la séquence (Sequence Understanding)

B. NER (Named Entity Recognition)

Exemple:

What is the capital of morocco and france?

Tokens: ['what', 'is', 'the', 'capital', 'of', 'morocco', 'and', 'france', '?']

Tags: ['O', 'O', 'O', 'O', 'O', 'B', 'O', 'B', 'O']

Entities: ['morocco', 'france']

04. 2. Compréhension de la séquence (Sequence Understanding)

B. NER (Named Entity Recognition)

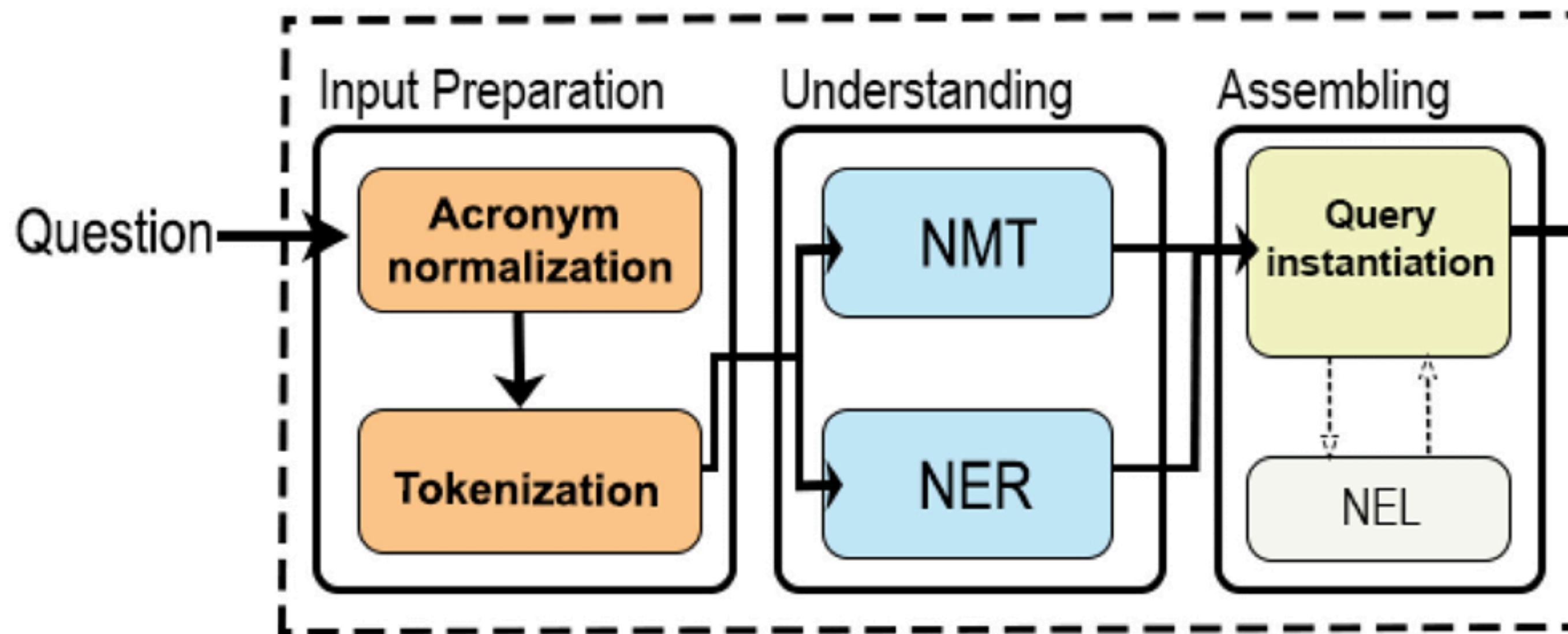
Exemple:

Entité reconnue :

"Mona Lisa"

3. Assemblage (Assembling)

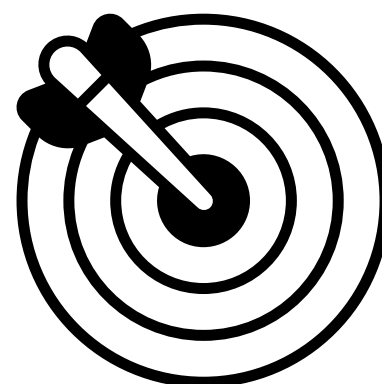
On remplace les placeholders du template par les entités du NER.



3. Assemblage (Assembling)

2 Méthodes:

But : On remplace les placeholders du template par les entités du NER.



04.

3. Assemblage (Assembling)

Méthode 1:

Exemple:

Template :

```
SELECT DISTINCT ?a WHERE { ?v dbo:instrument ?a . ?v rdfs:label $1 }
```

04.

3. Assemblage (Assembling)

Méthode 1:

Exemple:

NER trouve :

"Cat Stevens"

3. Assemblage (Assembling)

Méthode 1:

Exemple:

Résultat assemblé :

```
SELECT DISTINCT ?a WHERE {  
  ?v dbo:instrument ?a .  
  ?v rdfs:label "Cat Stevens"@en  
}
```

04.

3. Assemblage (Assembling)

Méthode 1:

Résultat dans DBpedia (SPARQL)

Cette requête risque de :

- ne rien retourner (si le label exact n'est pas présent)
- retourner un mauvais résultat
- dépendre d'une correspondance exacte du texte

Très fragile

3. Assemblage (Assembling)

Méthode 2:

CNEL — Étape 1 : Récupération des candidats NEL

Les systèmes Spotlight / WAT / REL renvoient généralement plusieurs entités pour “Mona Lisa”.

3. Assemblage (Assembling)

Méthode 2:

CNEL — Étape 1 : Récupération des candidats NEL

Spot détecté	Entité KB proposée
Mona Lisa	dbr:Mona_Lisa
The Mona Lisa	dbr:Mona_Lisa

3. Assemblage (Assembling)

Méthode 2:

CNEL — Étape 2 : On garde uniquement les spots qui chevauchent exactement l'entité NER

Spot	Chevauche ?	Garde ?
Mona Lisa	dbr:Mona_Lisa	✓ garder
The Mona Lisa	dbr:Mona_Lisa	✓ garder

04.

3. Assemblage (Assembling)

Méthode 2:

Il reste :

1. "Mona Lisa" → **dbr:Mona_Lisa**
2. "The Mona Lisa" → **dbr:The_Mona_Lisa**

04.

3. Assemblage (Assembling)

Méthode 2:

CNEL — Étape 3 : Calcul du Collision Index (CI)

Formule :

$$CI = | TE + TS - 2 \times OT |$$

- TE = tokens de l'entité NER → ["Mona", "Lisa"] = 2
- TS = tokens du spot
- OT = tokens qui se chevauchent

04.

3. Assemblage (Assembling)

Méthode 2:

CNEL — Étape 3 : Calcul du Collision Index (CI)

CANDIDAT 1 : Spot = "Mona Lisa"

- Spot tokens = ["Mona", "Lisa"] → TS = 2
- Overlap = ["Mona", "Lisa"] → OT = 2

$$\text{CI} = | 2 + 2 - 2 \times 2 | = | 4 - 4 | = 0 \text{ (parfait)}$$

04.

3. Assemblage (Assembling)

Méthode 2:

CNEL — Étape 3 : Calcul du Collision Index (CI)

CANDIDAT 2 : Spot = "The Mona Lisa"

- Spot tokens = ["The", "Mona", "Lisa"] → TS = 3
- Overlap = ["Mona", "Lisa"] → OT = 2

$$\text{CI} = | 2 + 3 - 2 \times 2 | = | 5 - 4 | = 1 \text{ (moins bon)}$$

04.

3. Assemblage (Assembling)

Méthode 2:

CNEL — Étape 4 : Score de confiance

CNEL aurait choisi celui avec le score de confiance le plus élevé.

- $CI(\text{"Mona Lisa"}) = 0 \rightarrow \text{meilleur}$
- $CI(\text{"The Mona Lisa"}) = 1$

Donc CNEL choisit : dbr:Mona_Lisa

04. 3. Assemblage (Assembling)

Méthode 2:

CNEL — Résultat final

Entité retenue :

dbr:Mona_Lisa

04.

3. Assemblage (Assembling)

Méthode 2:

CNEL a trouvé l'IRI correcte → alors le système ne garde PAS rdfs:label.

Il remplace toute la structure par l'IRI :

```
SELECT ?a WHERE {
```

```
<http://dbpedia.org/resource/Mona Lisa>    dbo:author    ?a .
```

```
}
```

3. Assemblage (Assembling)

Les deux Méthodes comparés :

Étape	CNEL KO	CNEL OK
Entité trouvée	“Mona Lisa”	“Mona Lisa”
CNEL	échoue	réussit
Valeur utilisée	"Mona Lisa"@en (texte brut)	dbr:Mona_Lisa (IRI KB)
Requête finale	faible, fragile	correcte, robuste
Résultat	peut être vide	Leonardo_da_Vinci

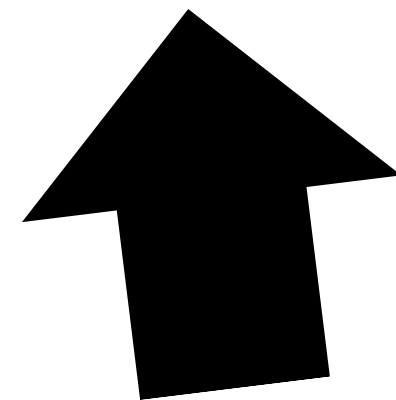
04. 4. Requête SPARQL (SPARQL Query)

Résultat réel dans DBpedia :

SPARQL | HTML5 table

a

http://dbpedia.org/resource/Leonardo_da_Vinci



05.

4. Implémentation

Dataset:

Dataset utilisée est LC-QUAD-1.0:

<https://github.com/AskNowQA/LC-QuAD>

05. 4. Implémentation

Dataset:

AskNowQA / LC-QuADPublic

<> Code

Issues13

Pull requests

Actions

Projects

Wiki

Security

Insights

data

7 Branches

0 Tags

Go to file

<> Code

mohnish-rygbee

Update readme with lcquad 2.0

9b8c16a · 5 years ago

117 Commits

.github/ISSUE_TEMPLATE	Added github issue templates here as well.	7 years ago
resources	Added aux files and updated links in README.	7 years ago
LICENSE.txt	Create LICENSE.txt	8 years ago
README.md	Update readme with lcquad 2.0	5 years ago
test-data.json	Changed UUIDs for sequential IDs, and changed the key `v...	7 years ago
train-data.json	Changed UUIDs for sequential IDs, and changed the key `v...	7 years ago

05. 5. Implémentation

Dataset:

Exemple:

_id "1501"

corrected_question "How many movies did Stanley Kubrick direct?"

intermediary_question "How many <movies> are there whose <director> is <Stanley Kubrick>?"

sparql_query "SELECT DISTINCT COUNT(?uri) WHERE {?uri <http://dbpedia.org/ontology/director> <http://dbpedia.org/resource/Stanley_Kubrick> .}"

sparql_template_id 101

05. 5. Implémentation

Normalisation des acronymes :

Country Converter (COCO)

IN: How large is the area of UK?

OUT: How large is the area of United Kingdom?

IN: GDP of the U.S. vs UAE and DRC?

OUT: GDP of the United States. vs United Arab Emirates and Democratic Republic of the Congo?

IN: Is EU larger than US?

OUT: Is European Union larger than United States?

IN: Flights from US to UK tomorrow.

OUT: Flights from United States to United Kingdom tomorrow.

05. 5. Implémentation

NER

```
Question (texte)
  |
  ▼
Tokenization (BertTokenizer)
  -> tokens (ex: [CLS], who, painted, the, mon, ##a, lisa, [SEP], [PAD], ...)
  -> input_ids (taille = MAX_LEN_Q)
  |
  ▼
Input IDs → BERT (TFBertModel, gelé / trainable=False)
              (sortie : sequence embeddings, shape = (MAX_LEN_Q, 768))
  |
  ▼
Bidirectional LSTM (return_sequences=True)
  (BiLSTM 128 units -> sortie shape = (MAX_LEN_Q, 256))
  |
  ▼
Dense(64, relu) -> Dropout(0.3)
  |
  ▼
Dense(3, softmax) ← sortie pour chaque token (classes = {0, B, I})
  (shape finale = (MAX_LEN_Q, 3))
  |
  ▼
Post-processing / grouping BIO → entités extra ↓ : (textes)
```

05. 5. Implémentation

NER

1. Entrée

- Question en texte libre.
- Le tokenizer BERT transforme la question en tokens et en `input_ids` (padded/truncated à `MAX_LEN_Q = 17`).

05. 5. Implémentation

NER

2. Encodage contextualisé (BERT)

- `TFBertModel.from_pretrained(...)` produit des embeddings contextuels pour chaque token.
- Dans ton code, `bert_model.trainable = False` → BERT est gelé (pas de mise à jour pendant l'entraînement).
- Sortie : tenseur (batch_size, MAX_LEN_Q, 768).

05. 5. Implémentation

NER

3. Séquence → BiLSTM

- Un Bidirectional LSTM lit la séquence (avant et arrière) pour capturer la dépendance contextuelle au niveau de la séquence (nom, début/fin d'entité, etc.).
- `return_sequences=True` parce qu'on veut une prédiction par token.

05. 5. Implémentation

NER

4. Projection & régularisation

- Dense(64, relu) réduit / transforme les features par token.
- Dropout(0.3) pour éviter l'overfitting.

05. 5. Implémentation

NER

5. Prédiction token-level (softmax)

- Dense(3, softmax) → pour chaque token, distribution sur les classes {O, B, I}.
- Loss utilisée : sparse_categorical_crossentropy (ton Y contient les ids 0/1/2).

05. 5. Implémentation

NER

6. Post-traitement (inference)

- `Dense(3, softmax)` → pour chaque token, distribution sur les classes {O, B, I}.
- Loss utilisée : `sparse_categorical_crossentropy` (ton Y contient les ids 0/1/2).

05. 5. Implémentation

NER

7. Entraînement

- Callbacks : EarlyStopping (sur val_accuracy), ReduceLROnPlateau, ModelCheckpoint.
- Batch size = 32, epochs=20

05. 5. Implémentation

NER

(taille, formes, labels)

- $\text{MAX_LEN_Q} = 17 \rightarrow$ toutes les séquences sont coupées / paddées à 17 tokens.
- BERT output size par token : 768 (couche finale).
- BiLSTM 128 $\rightarrow$ bidirectionnel $\rightarrow$ sortie par token dimension 256.
- Sortie finale par token : vecteur de taille 3 (probabilités pour O, B, I).
- Tags: 0=O, 1=B, 2=I.

05. 5. Implémentation

NER

<https://www.kaggle.com/code/yassintouzani/bert-dpedia/edit>

05. 5. Implémentation

NMT

```
Question (texte)
|
v
Tokenization (BertTokenizer)
-> tokens (padded/truncated à MAX_LEN_Q = 17)
-> input_ids (shape = (MAX_LEN_Q,))
|
v
BERT Encoder (TFBertModel, gelé)
-> sequence embeddings (batch, MAX_LEN_Q, 768)
-> [CLS] embedding = encoder_output[:,0,:] (utilisé pour init state)
|
v
Projection:
- encoder_projected = Dense(512)(encoder_output) # (batch, MAX_LEN_Q, 512)
- initial_state = tanh(Dense(512)([CLS])) # (batch, 512)
|
v
Decoder input (template tokens) – Pendant l'entraînement : teacher forcing
- template_input (padded, MAX_LEN_TEMPLATE = 80)
- template_embedding: Embedding(vocab_size+1, 300, mask_zero=True)
|
v
Decoder LSTM (512 units, return_sequences=True)
- initial_state = [initial_state, initial_state]
- decoder_output (batch, MAX_LEN_TEMPLATE, 512)
|
v
Attention (tf.keras.layers.Attention)
- attention_output = Attention([decoder_output, encoder_projected])
- (batch, MAX_LEN_TEMPLATE, 512)
|
v
Concatenate(decoder_output, attention_output) → (batch, MAX_LEN_TEMPLATE, 1024)
|
v
Dense(512, relu) → Dense(vocab_size+1, softmax)
-> output tokens prob. (batch, MAX_LEN_TEMPLATE, vocab_size+1)
|
v
Inference (greedy decoding / stop on [SEP] or [PAD])
```

05. 5. Implémentation

NMT

```
# Inputs
question_input = Input(shape=(MAX_LEN_Q,), dtype=tf.int32, name="question_input")
template_input = Input(shape=(MAX_LEN_TEMPLATE,), dtype=tf.int32, name="template_input")

# BERT Encoder
bert_encoder = BERTEncoder(bert_model, name="bert_encoder")
encoder_output = bert_encoder(question_input) # (batch, MAX_LEN_Q, 768)

# Project BERT output to decoder dimension
encoder_projected = Dense(512, name="encoder_projection")(encoder_output)

# Initialize decoder state from BERT [CLS] token
initial_state = Dense(512, activation='tanh', name="state_projection")(encoder_output[:, 0, :])

# Decoder - Embedding layer for template tokens
template_embedding = Embedding(
    input_dim=tokenizer.vocab_size + 1,
    output_dim=300,
    mask_zero=True,
    name="template_embedding"
)(template_input)

# Decoder LSTM
decoder_lstm = LSTM(512, return_sequences=True, return_state=True, name="decoder_lstm")
decoder_output, decoder_state = decoder_lstm(
```

05. 5. Implémentation

NMT

```
=====
TRAINING NMT MODEL
=====

Epoch 1/10
782/782  276s 338ms/step - accuracy: 0.1929 - loss: 2.6123 - val_accuracy: 0.3356 - val_loss: 0.4540 - learning_rate: 3.0000e-04

Epoch 2/10
782/782  260s 332ms/step - accuracy: 0.3519 - loss: 0.3101 - val_accuracy: 0.3491 - val_loss: 0.3149 - learning_rate: 3.0000e-04

Epoch 3/10
782/782  260s 333ms/step - accuracy: 0.3657 - loss: 0.1872 - val_accuracy: 0.3550 - val_loss: 0.2724 - learning_rate: 3.0000e-04

Epoch 4/10
782/782  260s 333ms/step - accuracy: 0.3717 - loss: 0.1361 - val_accuracy: 0.3560 - val_loss: 0.2697 - learning_rate: 3.0000e-04

Epoch 5/10
30/782  4:03 324ms/step - accuracy: 0.3700 - loss: 0.1126
```

05. 5. Implémentation

CNEL

Question: Who painted the Mona Lisa?

Template: `select distinct ?uri where {$1 dbo:author ?uri}`

NER Tags: O O O B I O

Linked 'mona lisa' -> http://dbpedia.org/resource/Mona_Lisa

Final SPARQL Query:

`select distinct ?uri where {<http://dbpedia.org/resource/Mona\_Lisa> dbo:author ?uri}`

05.

Voir une simulation sur Jupyter

SIMULATION

Merci pour votre attention !